TUTORIAL – 14
Selenium Web Automation

Aim

To create Selenium automation scripts in Python for browser navigation, form handling, web-element interaction, and validation testing.

Software / Tools Required

- PyCharm IDE
- Python 3.x
- Google Chrome browser
- Selenium WebDriver package
- Internet connection

Requirement / Problem Statement

Create a Selenium automation script that opens the SauceDemo website, handles the login form, enters a valid username and password, clicks the login button, validates the Products page, selects a product, opens the cart, and validates the URL/page navigation.

Procedure

- Install Python 3.x if it is not already installed. On Windows, enable “Add Python to PATH” during installation.
- Install and open PyCharm. Create a Python project named SoftwareTesting and select a valid Python 3.x interpreter.
- Create a Python file named tutorial14.py.

Open the PyCharm Terminal and install Selenium using: python -m pip install selenium. On Windows, py -m pip install selenium can also be used.

Paste the source code provided in this report into tutorial14.py and save the file.

Ensure Google Chrome is installed and internet access is available. Selenium Manager normally obtains the compatible ChromeDriver automatically.

Right-click tutorial14.py and select Run 'tutorial14', or click the green Run button.

Observe Chrome opening SauceDemo, submitting the demo credentials, selecting the Sauce Labs Backpack, and opening the cart.

Check the Run / Console window for PASS messages. If the script fails, review the error, verify the Python/Selenium/Chrome setup, correct the issue, and run it again.

Script / Source Code

File name: tutorial14.py

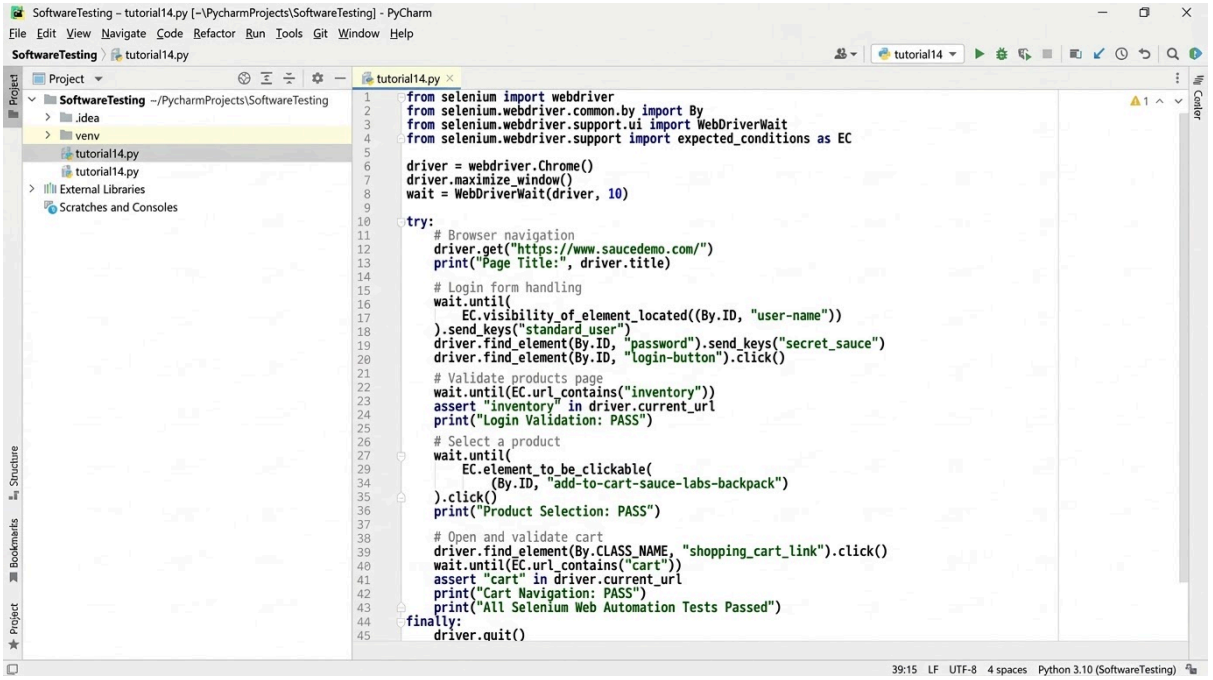
```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions
as EC
```

```
driver = webdriver.Chrome()
driver.maximize_window()
wait = WebDriverWait(driver, 10)
try:
  # Browser navigation
  driver.get("https://www.saucedemo.com/")
  print("Page Title:", driver.title)
  # Login form handling
  wait.until(
      EC.visibility_of_element_located((By.ID, "user-name"))
  ).send_keys("standard_user")
  driver.find_element(By.ID,
"password").send_keys("secret_sauce")
  driver.find_element(By.ID, "login-button").click()
  # Validate products page
  wait.until(EC.url_contains("inventory"))
  assert "inventory" in driver.current_url
  print("Login Validation: PASS")
  # Select a product
  wait.until(
      EC.element_to_be_clickable(
          (By.ID, "add-to-cart-sauce-labs-backpack")
      )
  ).click()
  print("Product Selection: PASS")
  # Open and validate cart
  driver.find_element(By.CLASS_NAME,
"shopping_cart_link").click()
  wait.until(EC.url_contains("cart"))
  assert "cart" in driver.current_url
  print("Cart Navigation: PASS")
  print("All Selenium Web Automation Tests Passed")
finally:
```

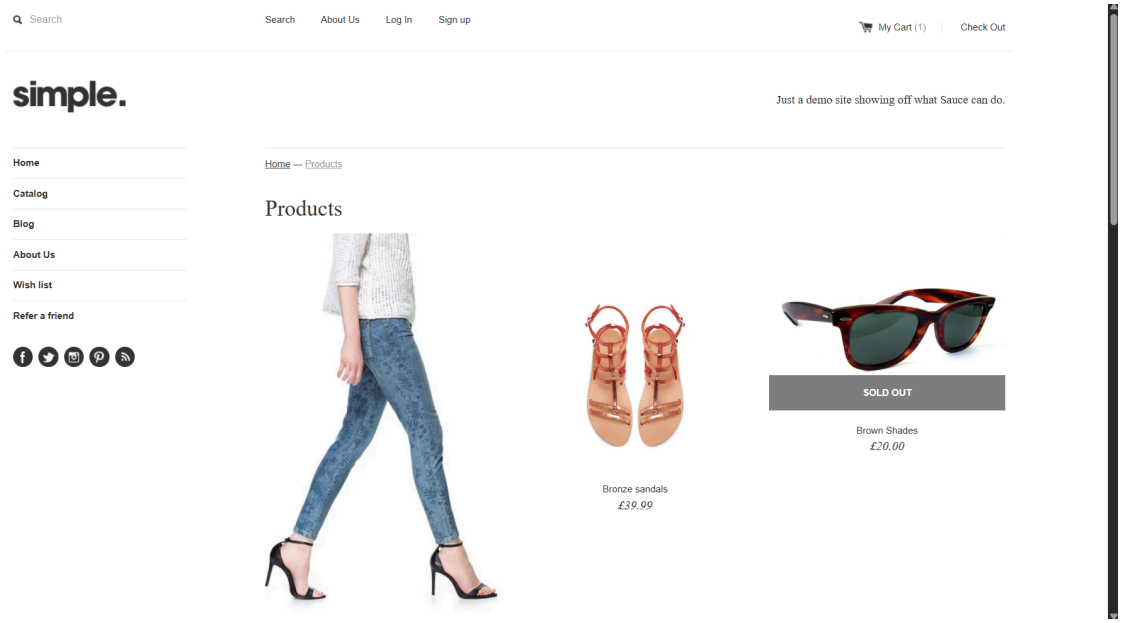
driver.quit()

Output Screenshot

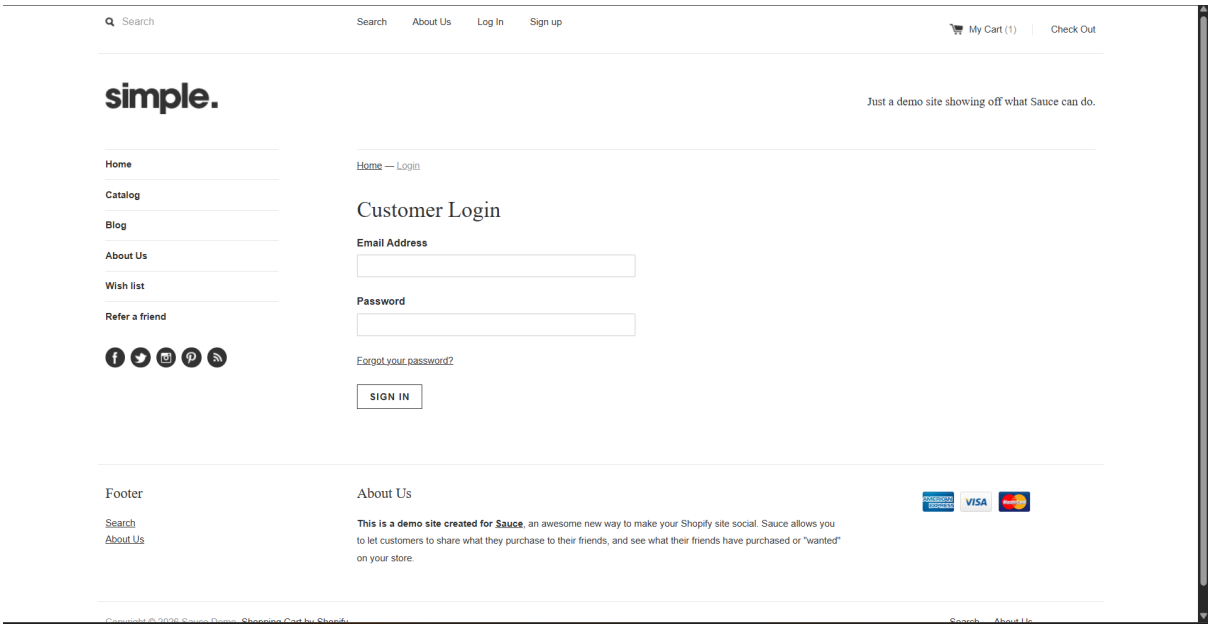
Screenshot 1 – Source Code in PyCharm



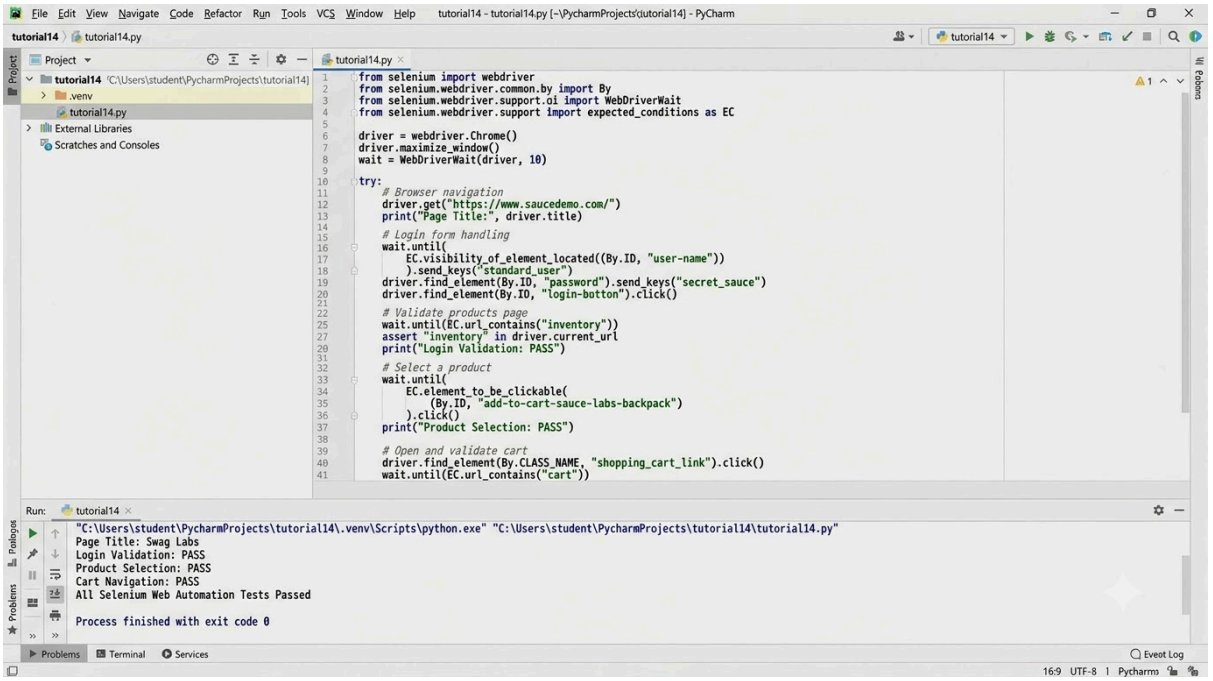
SCREENSHOT 2: Google Crome browser output



Screenshot 2 – Login Page



Screenshot 3 – PyCharm Run / Console Output



Expected Output

Page Title: Swag Labs

Login Validation: PASS

Product Selection: PASS

Cart Navigation: PASS

All Selenium Web Automation Tests Passed

Conclusion

This practical demonstrates browser automation using Python and Selenium WebDriver. The script opens SauceDemo, logs in with valid demo credentials, validates navigation to the Products page, adds the Sauce Labs Backpack to the cart, and checks that the cart URL is opened. The script should be executed in the local PyCharm environment, and actual screenshots and results should be recorded after successful execution.